

Lecture Scribe: Sequence Alignment and Edit Distance

CS G526 Advanced Algorithms & Complexity

Lecture 12 – 07/09/2026

Prof. Aniket Basu Roy

Student ID	Student Name	Student Email
2026H1030047G	Dhruv Rakeshbhai Patel	h20260047@goa.bits-pilani.ac.in
2026H1030045G	Sahil	h20260045@goa.bits-pilani.ac.in

1 Introduction

Sequence Alignment is a problem of finding how similar two strings are by matching their characters in the same order.

It has applications in **search engines and online dictionaries**, where a system may correct a misspelled word, and in **computational biology**, where DNA or other biological sequences are compared.

Let the two sequences be

$$X = x_1x_2 \dots x_m$$

and

$$Y = y_1y_2 \dots y_n.$$

We want to match characters of X with characters of Y . The matching must be **crossing-free**; such a matching is called an **alignment**.

Since different alignments are possible, we assign a cost to each alignment using a **gap penalty** and a **mismatch cost**. The objective is to find the alignment that minimizes the total cost.

2 Definitions, Utility Functions, and Terms

2.1 Sequence

A sequence is an ordered list of characters.

$$X = x_1x_2 \dots x_m, \quad Y = y_1y_2 \dots y_n$$

where

$$x_i, y_j \in \Sigma.$$

Here, Σ is the alphabet from which the characters are taken.

2.2 Matching

A matching M is a set of pairs

$$(i, j),$$

where (i, j) means that x_i is matched with y_j .

2.3 Crossing-Free Matching

A matching is crossing-free if the order of the matched characters is preserved.

If

$$(i, j), (i', j') \in M$$

and

$$i < i',$$

then we must have

$$j < j'.$$

Therefore,

$$\boxed{i < i' \implies j < j'}$$

A crossing-free matching is called an **alignment**.

2.4 Alignment

An **alignment** is a crossing-free matching between the two sequences.

For example, consider

$$X = ABC, \quad Y = BAC.$$

A possible alignment is

$$M = \{(1, 2), (3, 3)\}.$$

Here,

$$A \leftrightarrow A, \quad C \leftrightarrow C.$$

The character B in each sequence is left unmatched.

2.5 Gap Penalty

Let

$$\delta > 0$$

be the gap penalty.

For every unmatched character, we pay a cost of δ .

If k characters are unmatched, then

$$\text{GapPenalty}(M) = k\delta$$

For example, if

$$\delta = 2$$

and two characters are unmatched, then

$$\text{GapPenalty}(M) = 2(2) = 4.$$

2.6 Mismatch Cost

The mismatch cost function is

$$\alpha : \Sigma \times \Sigma \rightarrow \mathbb{R}$$

For the simple case used in the lecture,

$$\alpha(x_i, y_j) = \begin{cases} 0, & x_i = y_j, \\ 1, & x_i \neq y_j. \end{cases}$$

Thus, matching equal characters has cost 0, while matching different characters has cost 1.

The function can also be more general. Different pairs of characters may have different costs depending on the application.

2.7 Mismatch Cost of an Alignment

For an alignment M , the total mismatch cost is

$$\text{MismatchCost}(M) = \sum_{(i,j) \in M} \alpha(x_i, y_j)$$

2.8 Edit Distance / Total Cost

The total cost of an alignment is

$$\text{EditDistance}(M) = \text{GapPenalty}(M) + \text{MismatchCost}(M)$$

or

$$\text{EditDistance}(M) = \delta \cdot \#(\text{unmatched characters}) + \sum_{(i,j) \in M} \alpha(x_i, y_j)$$

The goal is to find the alignment having minimum total cost.

2.9 Dynamic Programming Function $OPT(i, j)$

Define

$$OPT(i, j)$$

as the minimum cost of aligning

$$X[1 \dots i]$$

with

$$Y[1 \dots j].$$

Therefore,

$$OPT(m, n)$$

gives the minimum cost for the complete two sequences.

3 Problem Definition

Given:

- two sequences

$$X = x_1 x_2 \dots x_m$$

and

$$Y = y_1 y_2 \dots y_n,$$

- a gap penalty parameter

$$\delta > 0,$$

- a mismatch cost function

$$\alpha : \Sigma \times \Sigma \rightarrow \mathbb{R},$$

find a crossing-free matching M that minimizes

$$\delta \cdot \#(\text{unmatched characters}) + \sum_{(i,j) \in M} \alpha(x_i, y_j).$$

Therefore, the required output is:

An alignment minimizing the total alignment cost.

There can be many candidate alignments, and we want the one with the minimum cost.

4 Examples of Candidate Alignments

Consider

$$X = ABC$$

and

$$Y = BAC.$$

Several crossing-free matchings are possible.

4.1 Alignment M_1

$$M_1 = \{(1, 1), (2, 2), (3, 3)\}.$$

This matches:

$$A \leftrightarrow B, \quad B \leftrightarrow A, \quad C \leftrightarrow C.$$

There are two mismatches and one exact match.

4.2 Alignment M_2

$$M_2 = \{(1, 2), (3, 3)\}.$$

This matches:

$$A \leftrightarrow A, \quad C \leftrightarrow C.$$

The B in X and the B in Y are left unmatched.

Therefore, there are two gaps.

4.3 Alignment M_3

$$M_3 = \{(2, 1), (3, 3)\}.$$

This matches:

$$B \leftrightarrow B, \quad C \leftrightarrow C.$$

Again, two characters remain unmatched.

These examples show that simply maximizing the number of matched pairs is not enough. We also have to consider mismatches and unmatched characters.

5 Finding the Optimal Alignment

We want to find the alignment with minimum cost.

Let

$$OPT(i, j)$$

denote the minimum cost of aligning

$$X[1 \dots i]$$

with

$$Y[1 \dots j].$$

So, the final answer will be

$$\boxed{OPT(m, n)}.$$

To find $OPT(i, j)$, look at the last characters:

$$x_i \quad \text{and} \quad y_j.$$

There are three possible cases.

5.1 Case 1: x_i and y_j are matched

If x_i is matched with y_j , then the remaining problem is

$$X[1 \dots i - 1]$$

and

$$Y[1 \dots j - 1].$$

Therefore, the cost is

$$\boxed{OPT(i - 1, j - 1) + \alpha(x_i, y_j)}$$

If

$$x_i = y_j,$$

then

$$\alpha(x_i, y_j) = 0.$$

Otherwise, the mismatch cost is added.

5.2 Case 2: x_i is unmatched

Here x_i is left as a gap.

So, we pay one gap penalty δ , and the remaining problem is

$$X[1 \dots i - 1]$$

and

$$Y[1 \dots j].$$

Hence,

$$\boxed{OPT(i-1, j) + \delta}$$

5.3 Case 3: y_j is unmatched

Similarly, if y_j is left unmatched, we pay δ .

The remaining problem is

$$X[1 \dots i]$$

and

$$Y[1 \dots j-1].$$

Hence,

$$\boxed{OPT(i, j-1) + \delta}$$

5.4 Recurrence

We take the minimum of the three cases:

$$\boxed{OPT(i, j) = \min \begin{cases} OPT(i-1, j-1) + \alpha(x_i, y_j), \\ OPT(i-1, j) + \delta, \\ OPT(i, j-1) + \delta. \end{cases}}$$

So, at every cell, we ask:

Should we match the two characters, or leave one of them unmatched?

We choose the option with the minimum cost.

6 Dynamic Programming Recurrence

Combining the three possibilities gives:

$$\boxed{OPT(i, j) = \min \begin{cases} OPT(i-1, j-1) + \alpha(x_i, y_j), \\ OPT(i-1, j) + \delta, \\ OPT(i, j-1) + \delta. \end{cases}}$$

For the simple mismatch-cost function,

$$\alpha(x_i, y_j) = \begin{cases} 0, & x_i = y_j, \\ 1, & x_i \neq y_j. \end{cases}$$

Thus, when

$$x_i = y_j,$$

we have

$$OPT(i, j) = \min \{OPT(i-1, j-1), OPT(i-1, j) + \delta, OPT(i, j-1) + \delta\}.$$

The three transitions are:

Transition	Meaning
$(i-1, j-1)$	Match x_i with y_j
$(i-1, j)$	Leave x_i unmatched
$(i, j-1)$	Leave y_j unmatched

7 Base Cases

Before filling the DP table, we need to initialize the first row and first column.

Recall that

$$OPT(i, j)$$

is the minimum cost of aligning

$$X[1 \dots i]$$

with

$$Y[1 \dots j].$$

7.1 $OPT(0, 0)$

If both sequences are empty, there is nothing to match.

Therefore,

$$\boxed{OPT(0, 0) = 0}$$

7.2 $OPT(i, 0)$

Here, the second sequence is empty:

$$Y[1 \dots 0] = \epsilon.$$

The i characters of X cannot be matched with anything, so all of them have to be left as gaps.

Since every gap costs δ ,

$$\boxed{OPT(i, 0) = i\delta}$$

For example,

$$OPT(3, 0) = 3\delta.$$

7.3 $OPT(0, j)$

Similarly, if the first sequence is empty, all j characters of Y have to be left unmatched.

Therefore,

$$\boxed{OPT(0, j) = j\delta}$$

For example,

$$OPT(0, 4) = 4\delta.$$

7.4 Initial DP Table

The first row and first column are therefore:

$X \setminus Y$	ϵ	y_1	y_2	$\cdots$	y_n
ϵ	0	δ	2δ	$\cdots$	$n\delta$
x_1	δ				
x_2	2δ				
$\vdots$	$\vdots$				
x_m	$m\delta$				

After this initialization, the remaining cells are filled using the recurrence from Section 6.

8 Proof of Correctness

The recurrence is based on the last characters of the two sequences.

Let

$$OPT(i, j)$$

be the minimum cost of aligning

$$X[1 \dots i]$$

with

$$Y[1 \dots j].$$

Consider an optimal alignment for these two prefixes.

8.1 Case 1: x_i is matched with y_j

If the last two characters are matched, then the characters before them form an alignment between

$$X[1 \dots i-1]$$

and

$$Y[1 \dots j-1].$$

Therefore, the cost is

$$OPT(i-1, j-1) + \alpha(x_i, y_j).$$

The smaller alignment must also be optimal.

If there were a cheaper alignment for these two prefixes, we could replace the current one with it and get a cheaper alignment for the complete problem. This would contradict the optimality of $OPT(i, j)$.

8.2 Case 2: x_i is left unmatched

If x_i is not matched with a character of Y , we pay one gap penalty:

$$\delta.$$

The remaining problem is

$$X[1 \dots i-1]$$

with

$$Y[1 \dots j].$$

Therefore, the cost is

$$OPT(i-1, j) + \delta.$$

Again, the smaller alignment must be optimal; otherwise, we could replace it with a cheaper one.

8.3 Case 3: y_j is left unmatched

Similarly, if y_j is unmatched, we pay

$$\delta.$$

The remaining problem is

$$X[1 \dots i]$$

with

$$Y[1 \dots j-1].$$

Therefore, the cost is

$$OPT(i, j - 1) + \delta.$$

8.4 Combining the Cases

For the last characters, these are the three cases that need to be considered:

$$OPT(i, j) = \min \begin{cases} OPT(i - 1, j - 1) + \alpha(x_i, y_j), \\ OPT(i - 1, j) + \delta, \\ OPT(i, j - 1) + \delta. \end{cases}$$

Since an optimal alignment must use one of these cases, taking the minimum gives the minimum possible cost.

Hence, the recurrence correctly computes $OPT(i, j)$, and therefore

$$OPT(m, n)$$

gives the minimum cost alignment of the two complete sequences.

□

9 Dynamic Programming Table and Algorithm

We store the values of $OPT(i, j)$ in a two-dimensional table.

Here,

$$OPT(i, j)$$

represents the minimum cost of aligning

$$X[1 \dots i]$$

with

$$Y[1 \dots j].$$

Therefore, the table has

$$(m + 1) \times (n + 1)$$

entries.

The first row and first column are initialized using

$$OPT(i, 0) = i\delta$$

and

$$OPT(0, j) = j\delta.$$

The remaining cells are filled using

$$OPT(i, j) = \min \begin{cases} OPT(i-1, j-1) + \alpha(x_i, y_j), \\ OPT(i-1, j) + \delta, \\ OPT(i, j-1) + \delta. \end{cases}$$

9.1 DP Table Structure

The rows correspond to the characters of X , and the columns correspond to the characters of Y .

$X \backslash Y$	ϵ	y_1	y_2	$\cdots$	y_{n-1}	y_n
ϵ	0	δ	2δ	$\cdots$	$(n-1)\delta$	$n\delta$
x_1	δ					
x_2	2δ					
$\vdots$	$\vdots$					
x_m	$m\delta$					$OPT(m, n)$

Figure 1: Dynamic programming table for sequence alignment.

Each cell considers three possible moves:

Move	Meaning
Diagonal $(i-1, j-1)$	Match x_i with y_j
Up $(i-1, j)$	Leave x_i unmatched
Left $(i, j-1)$	Leave y_j unmatched

The minimum of these three choices is stored in the current cell.

9.2 Pseudocode

```

SEQUENCE-ALIGNMENT(X, Y, delta, alpha)

  m <- length(X)
  n <- length(Y)

  create OPT[0..m][0..n]

  OPT[0][0] <- 0

  for i <- 1 to m do
    OPT[i][0] <- i * delta

  for j <- 1 to n do
    OPT[0][j] <- j * delta

  for i <- 1 to m do
    for j <- 1 to n do

      matchCost <- alpha(X[i], Y[j])

      OPT[i][j] <- min(
        OPT[i-1][j-1] + matchCost,

```

```

        OPT[i-1][j] + delta,
        OPT[i][j-1] + delta
    )

    return OPT[m][n]

```

The algorithm first initializes the first row and column and then fills the table from left to right and top to bottom.

Finally,

$$\boxed{OPT(m, n)}$$

gives the minimum alignment cost.

9.3 Recovering the Actual Alignment

The DP table gives the minimum cost. We can also recover the actual alignment by performing a **traceback**.

We start from

$$OPT(m, n)$$

and move backwards until we reach

$$OPT(0, 0).$$

At each cell:

- A diagonal move means x_i is matched with y_j .
- An upward move means x_i is left unmatched.
- A left move means y_j is left unmatched.

The characters are collected while moving backwards. After the traceback, the obtained sequences are reversed to get the final alignment.

9.4 Traceback Pseudocode

```

RECONSTRUCT-ALIGNMENT(X, Y, OPT, delta, alpha)

    i <- length(X)
    j <- length(Y)

    A_X <- empty sequence
    A_Y <- empty sequence

    while i > 0 or j > 0 do
        if i > 0 and j > 0 and
           OPT[i][j] ==
           OPT[i-1][j-1] + alpha(X[i], Y[j]) then

            prepend X[i] to A_X
            prepend Y[j] to A_Y

```

```

        i <- i - 1
        j <- j - 1

    else if i > 0 and
        OPT[i][j] ==
        OPT[i-1][j] + delta then

        prepend X[i] to A_X
        prepend '-' to A_Y

        i <- i - 1

    else

        prepend '-' to A_X
        prepend Y[j] to A_Y

        j <- j - 1

    return A_X, A_Y

```

If more than one transition gives the same minimum value, then more than one optimal alignment may exist.

10 Worked Example using the DP Table

Consider the two sequences

$$X = ABC$$

and

$$Y = BAC.$$

For this example, let

$$\delta = 1$$

and use the following mismatch-cost function:

$$\alpha(x_i, y_j) = \begin{cases} 0, & x_i = y_j, \\ 3, & x_i \neq y_j. \end{cases}$$

This means that a mismatch costs 3, while introducing a gap costs 1.

10.1 Initialization

Using

$$OPT(i, 0) = i\delta$$

and

$$OPT(0, j) = j\delta,$$

the initial values are:

$X \backslash Y$	ϵ	B	A	C
ϵ	0	1	2	3
A	1			
B	2			
C	3			

10.2 Computing the DP Table

The completed table is:

$X \backslash Y$	ϵ	B	A	C
ϵ	0	1	2	3
A	1	2	1	2
B	2	1	2	3
C	3	2	3	2

Figure 2: Completed DP table for $X = ABC$ and $Y = BAC$.

Therefore,

$$\boxed{OPT(3, 3) = 2}$$

10.3 Calculating a DP Cell

Consider

$$OPT(2, 2),$$

where

$$x_2 = B, \quad y_2 = A.$$

Since

$$B \neq A,$$

we have

$$\alpha(B, A) = 3.$$

The three possible costs are:

$$OPT(1, 1) + 3 = 1 + 3 = 4,$$

$$OPT(1, 2) + 1 = 1 + 1 = 2,$$

and

$$OPT(2, 1) + 1 = 1 + 1 = 2.$$

Therefore,

$$\boxed{OPT(2, 2) = \min(4, 2, 2) = 2}$$

This shows how each DP cell is obtained from the three previously calculated values.

10.4 Traceback

Start from

$$OPT(3, 3) = 2.$$

At this position,

$$C = C,$$

so the diagonal transition is chosen:

$$(3, 3) \rightarrow (2, 2).$$

At

$$OPT(2, 2),$$

we have

$$B \neq A.$$

A direct mismatch would cost 3, while introducing gaps gives a smaller cost.

One optimal alignment is:

$$\begin{array}{cccc} - & A & B & C \\ B & A & - & C \end{array}$$

Therefore,

$$\boxed{X' = -ABC}$$

and

$$\boxed{Y' = BA - C}$$

There are two gaps, so the gap cost is

$$2(1) = 2.$$

The exact matches are

$$A \leftrightarrow A$$

and

$$C \leftrightarrow C,$$

so their mismatch costs are zero.

Therefore,

$$\boxed{\text{Total Cost} = 2}$$

which agrees with

$$\boxed{OPT(3, 3) = 2.}$$

10.5 Traceback Directions

The traceback directions can be represented as:

	ϵ	B	A	C
ϵ	0	$\leftarrow$	$\leftarrow$	$\leftarrow$
A	$\uparrow$		$\nwarrow$	
B	$\uparrow$	$\nwarrow$	$\uparrow / \leftarrow$	
C	$\uparrow$			$\nwarrow$

where:

$\nwarrow$ = match/mismatch

$\uparrow$ = gap in Y

and

$\leftarrow$ = gap in X .

Following a minimum-cost path from the bottom-right cell to the top-left cell gives an optimal alignment.

11 Effect of the Cost Parameters

The choice of δ and α affects which alignment is considered optimal.

For example, suppose

$$\delta = 2$$

and

$$\alpha(x, y) = \begin{cases} 0, & x = y, \\ 1, & x \neq y. \end{cases}$$

For

$$X = ABC, \quad Y = BAC,$$

the direct alignment

$$A \leftrightarrow B, \quad B \leftrightarrow A, \quad C \leftrightarrow C$$

has two mismatches.

Therefore,

$$\text{Cost} = 1 + 1 = 2.$$

Introducing two gaps would cost

$$2 + 2 = 4.$$

Hence, under this scoring system, the mismatch-based alignment is preferred.

Therefore:

The optimal alignment depends on the scoring model.

This is why both δ and α are part of the input to the problem.

12 Time Complexity

The DP table contains

$$(m + 1)(n + 1)$$

entries.

Therefore, the number of states is

$$\Theta(mn).$$

For every state, we calculate three candidate values and take their minimum.

Each state takes

$$O(1)$$

time.

Hence,

$$\boxed{T(m, n) = O(mn)}$$

Thus, the time complexity of the sequence-alignment dynamic programming algorithm is

$$O(mn).$$

13 Space Complexity

The complete DP table contains

$$(m + 1)(n + 1)$$

entries.

Therefore, the standard implementation uses

$$O(mn)$$

space.

If only the minimum cost is required, we do not need to store the complete table. Since each cell depends only on the previous row and current row, the space can be reduced to

$$O(\min(m, n))$$

using rolling arrays.

However, if the actual alignment is required using the traceback procedure, keeping the full DP table is the simpler approach.

14 Key Takeaways

Sequence Alignment can be summarized as:

$$\text{Matching} \rightarrow \text{Crossing-free Matching} \rightarrow \text{Alignment} \rightarrow \text{Cost} \rightarrow \text{Dynamic Programming}$$

The main definitions are:

$$\text{GapPenalty}(M) = \delta \cdot \#(\text{unmatched characters})$$

$$\text{MismatchCost}(M) = \sum_{(i,j) \in M} \alpha(x_i, y_j)$$

and

$$\text{EditDistance}(M) = \text{GapPenalty}(M) + \text{MismatchCost}(M)$$

The DP function is:

$$OPT(i, j) = \text{minimum cost of aligning } X[1 \dots i] \text{ with } Y[1 \dots j]$$

with base cases:

$$OPT(0, 0) = 0$$

$$OPT(i, 0) = i\delta$$

$$OPT(0, j) = j\delta$$

and recurrence:

$$OPT(i, j) = \min \begin{cases} OPT(i-1, j-1) + \alpha(x_i, y_j), \\ OPT(i-1, j) + \delta, \\ OPT(i, j-1) + \delta. \end{cases}$$

The final answer is:

$$OPT(m, n)$$

and the standard dynamic programming implementation has:

$$\text{Time} = O(mn)$$

and

$$\text{Space} = O(mn).$$

Sources Used

1. **Prof. Aniket Basu Roy**, “CSG 526 – Advanced Algorithms and Complexity, Lecture 12,” 07/09/2026.
2. **GeeksforGeeks**, “Sequence Alignment Problem,” <https://www.geeksforgeeks.org/dsa/sequence-alignment-problem/>
3. **Stack Overflow**, “Sequence Alignment with Dynamic Programming,” <https://stackoverflow.com/questions/33585107/sequence-alignment-with-dynamic-programming>
4. **GitHub**, **noporpoise/seq-align**, “Sequence alignment implementations including Needleman–Wunsch and Smith–Waterman,” <https://github.com/noporpoise/seq-align>
5. **GitHub**, **skotchandsoda/needleman-wunsch**, “Needleman–Wunsch sequence alignment implementation,” <https://github.com/skotchandsoda/needleman-wunsch>